In [4]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import mysql.connector
import warning
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="******",
    database="globaldb"
)
q_t = "SHOW TABLES"
table = pd.read_sql(q_t,conn)

for i in table.iloc[:, 0]:
    df = pd.read_sql(f"SELECT * FROM `{i}`", conn)
    globals()[f"df_{i}"] = df
    print(f"Created dataframe: df_{i}")
conn.close()

df_db_customer.tail()
del df_db_customer['pincode']
del df_db_customer['interests']
df_db_customer.rename(columns = {'name':'Customer Name'},inplace = True)
```

```
Created dataframe: df_db_customer
Created dataframe: df_db_subscription
Created dataframe: df_db_support
C:\Users\divya\AppData\Local\Temp\ipykernel_24124\680818458.py:13: UserWarning: p
andas only supports SQLAlchemy connectable (engine/connection) or database string
URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please con
sider using SQLAlchemy.
  table = pd.read_sql(q_t,conn)
C:\Users\divya\AppData\Local\Temp\ipykernel_24124\680818458.py:16: UserWarning: p
andas only supports SQLAlchemy connectable (engine/connection) or database string
URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please con
sider using SQLAlchemy.
  df = pd.read_sql(f"SELECT * FROM `{i}`", conn)
```

In [6]:
```
df_db_customer.rename(columns = {'name':'Customer Name'},inplace = True)
df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

In [9]:
```
df_db_customer['gender'].unique()
df_db_customer['gender'] = df_db_customer['gender'].apply(lambda x:'Male' if str
df_db_customer['country'] = df_db_customer.groupby(['state'])['country'].transfo
df_db_customer[df_db_customer['country'].isna()]
```

Out[9]:

customerid	Customer Name	country	state	gender	dob

In [10]:
```
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 4188 entries, 0 to 4187
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   customerid              4188 non-null   str
 1   subscription_start_date 4188 non-null   str
 2   subscription_type       4188 non-null   str
 3   renewal_date            4188 non-null   str
 4   plan_type               4188 non-null   str
 5   contract_type           4188 non-null   str
 6   cancellation_date       4188 non-null   str
 7   cancellation_reason     4188 non-null   str
 8   monthly_charges         4188 non-null   float64
 9   cltv                    4188 non-null   int64
 10  churn_score             4188 non-null   int64
dtypes: float64(1), int64(2), str(8)
memory usage: 597.3 KB
```

In [11]:
```
df_db_subscription[['subscription_start_date', 'cancellation_date']] = df_db_sub
```

In [12]:
```
df_db_subscription.head()
```

Out[12]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
0	0002-ORFBO	2020-05-14	Refferal	2025-05-14	Premium	
1	0003-MKNFE	2020-03-19	Refferal	2024-03-19	Premium	
2	0004-TLHLJ	2020-02-16	Refferal	2025-02-16	Premium	
3	0011-IGKFF	2020-11-25	Organic	2024-11-25	Standard	
4	0013-EXCHZ	2022-05-12	Paid	2024-05-12	Standard	

In [13]:
```
df_db_subscription[df_db_subscription['cancellation_date'].isna()].tail()
```

Out[13]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	c
4182	4367-YTFMO	2022-08-14	Refferal	2024-08-14	Standard	
4183	4368-VVYLC	2019-12-26	Paid	2025-12-26	Standard	
4185	4371-AXGHY	2021-09-29	Paid	2024-09-29	Standard	
4186	4371-IPLFG	2019-04-21	Paid	2025-04-21	Premium	
4187	4371-HOKSI	2022-11-18	Refferal	2024-11-18	Standard	

In [14]:
```
df_db_support.tail()
```

Out[14]:

	customerid	complaint_date	escalations	csat_score	col_1	comment
1757	4354-PCVKH	2024-02-04 00:00:00	Y	30		service issue
1758	4365-ABXUA	2025-07-08 00:00:00	N	50		service issue
1759	4365-ABXUA	2025-07-09 00:00:00	N	30		service issue
1760	4368-BCSLB	2025-11-03 00:00:00	Y	15		
1761	4371-AXGHY	2024-09-26 00:00:00	Y	20		

In [15]:
```
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1762 entries, 0 to 1761
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      1762 non-null   str
 1   complaint_date  1762 non-null   str
 2   escalations     1762 non-null   str
 3   csat_score      1762 non-null   int64
 4   col_1           1762 non-null   str
 5   comment         1762 non-null   str
dtypes: int64(1), str(5)
memory usage: 148.1 KB
```

In [16]:
```
df_db_support.drop(columns = ['comment'],inplace = True)
df_db_subscription['is_cancelled'] = df_db_subscription['cancellation_date'].not
    True: 1,
    False: 0
})
```

In [213…
```
df_sub.shape
```

Out[213… (4188, 17)

In [214…
```
df_temp = df_db_support.copy()
```

In [215…
```
df_temp['support_count'] = (
    df_temp.groupby('customerid')['customerid'].transform('count')
)
```

In [216…
```
df_temp = df_temp.drop_duplicates(
    subset='customerid',
    keep='last'
)
```

In [217…
```
df = df_db_subscription.merge(
    df_db_customer,
    how='left',
    on='customerid'
).merge(
    df_temp,
    how='left',
    on='customerid'
)
```

```
print(df.shape)
print(df['customerid'].nunique())
```

```
(4188, 22)
4188
```

In [218…
```
df.to_csv('Final.csv',index=False)
```

In [219…
```
df.columns
```

Out[219…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'is_cancelled', 'Customer Name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'col_1',
       'support_count'],
      dtype='str')
```

In [220…
```
churn_rate = df['is_cancelled'].mean()*100
print("Churn Rate :",round(churn_rate,2),'%')
retaintaion_rate = 100 - churn_rate
print("Retaintaion Rate :",round(retaintaion_rate,2),'%')
```

```
Churn Rate : 28.15 %
Retaintaion Rate : 71.85 %
```

In [221…
```
churn_plan = df.groupby('plan_type')['is_cancelled'].mean()
```

In [222…
```
churn_contract = (
    df.groupby('contract_type')['is_cancelled']
      .mean()
      .sort_values(ascending=False)
)

print(churn_contract)
```

```
contract_type
Monthly    0.293031
Annual     0.272014
Name: is_cancelled, dtype: float64
```

In [223…
```
Today = pd.Timestamp.today()
df['Tenure_date'] = np.where(
    df['is_cancelled'] !=0,(df['cancellation_date'] - df['subscription_start_dat
)
```

In [224…
```
print("Average Tenure (days):", int(avg_tenure))
print("Avg tenure (in days) :",avg_tenure)
#escalation rate
esc_rate = (df['escalations'].str.startswith('Y')).mean()*100
print("escalations rate:",esc_rate.round(2),"%")
df['escalation_flag']=np.where(
    df['escalations'] == 'Y',1,0
)
corr_df = df[['escalation_flag','is_cancelled']].dropna()
correlation = corr_df['escalation_flag'].corr(corr_df['is_cancelled'])
print('correlation is ',round(correlation*100,2),'%')
condition =[
    (df['churn_score'] <50),
    (df['churn_score'] >=50) & (df['churn_score'] <75) ,
```

```
    (df['churn_score'] >=75)
]
choice = ['low','med','high']
df['Churn_risk'] = np.select(condition,choice,default = "Unknown")
df2 = df.copy()
```

```
Average Tenure (days): 1752
Avg tenure (in days) : 1752.0
escalations rate: 15.64 %
correlation is  44.67 %
```

In [225…

```
risk_summary = (
    df.groupby('Churn_risk')
      .agg(
          customers=('customerid', 'nunique'),
          churn_rate=('is_cancelled', 'mean'),
          revenue=('monthly_charges', 'sum'),
          cltv=('cltv', 'sum')
      )
)

risk_summary['churn_rate'] *= 100

risk_summary
```

Out[225…

Churn_risk	customers	churn_rate	revenue	cltv
high	1179	100.0	19483.21	730645
low	2279	0.0	38712.21	2823301
med	730	0.0	12185.70	911936

In [226…

```
df3 = df2.set_index(['plan_type','contract_type']).sort_values(by=['plan_type','
```

In [227…

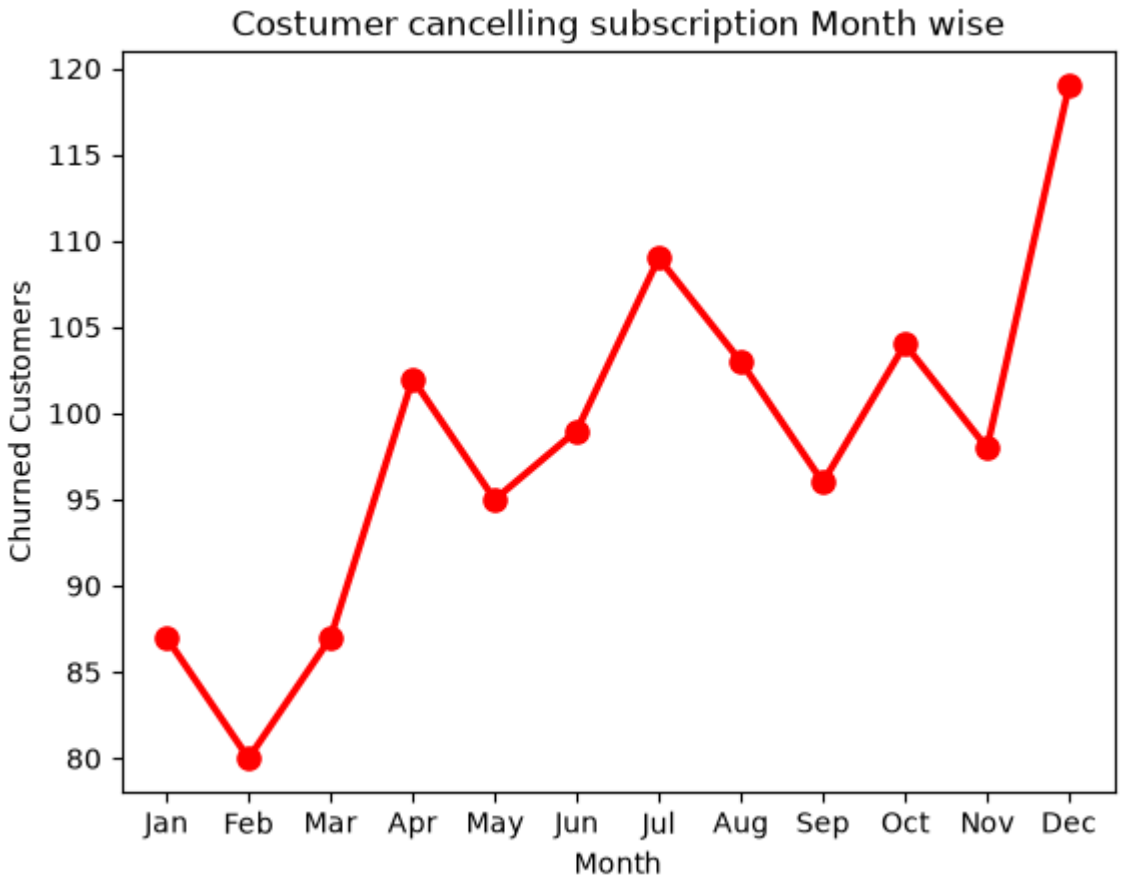
```
df2['cancellation month'] = df2['cancellation_date'].dt.to_period('M').value_cou

churn_trend = (
    df2[df2['is_cancelled'] == 1]['cancellation_date'].dt.month.value_counts().s
)

plt.plot(churn_trend.index,churn_trend.values,marker='o',markersize=8,linewidth=

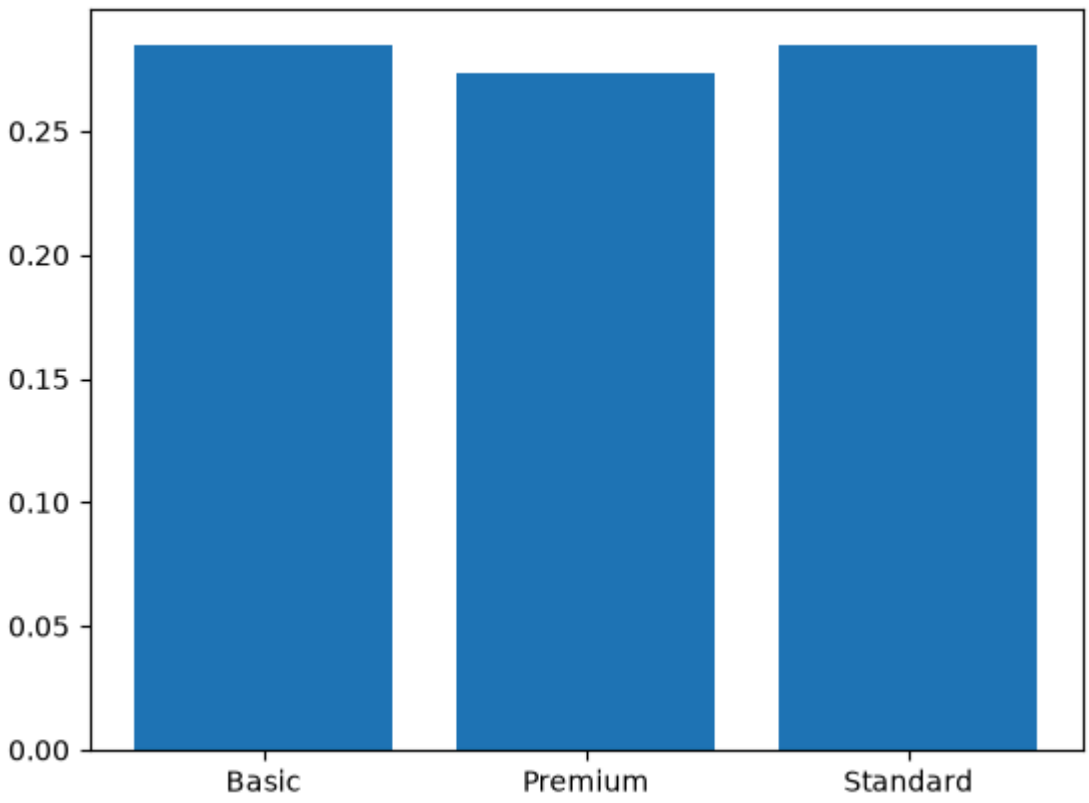
plt.xticks(
    range(1, 13),
    ['Jan','Feb','Mar','Apr','May','Jun',
     'Jul','Aug','Sep','Oct','Nov','Dec']
)

plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.title('Costumer cancelling subscription Month wise')
plt.show()
```



In [228…
```
plt.bar(churn_plan.index,churn_plan.values)
```

Out[228…
```
<BarContainer object of 3 artists>
```



In [229…
```
churn_plan
```

Out[229…
```
plan_type
Basic       0.284804
Premium     0.273819
Standard    0.284783
Name: is_cancelled, dtype: float64
```

In [230…
```
df2.columns
```

Out[230…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'is_cancelled', 'Customer Name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'col_1', 'support_count',
       'Tenure_date', 'escalation_flag', 'Churn_risk', 'cancellation month'],
      dtype='str')
```

In [231…
```
churn_country = df2.groupby('country')['is_cancelled'].mean().sort_values()

colors = plt.cm.Set2(np.linspace(0, 1, len(churn_country)))

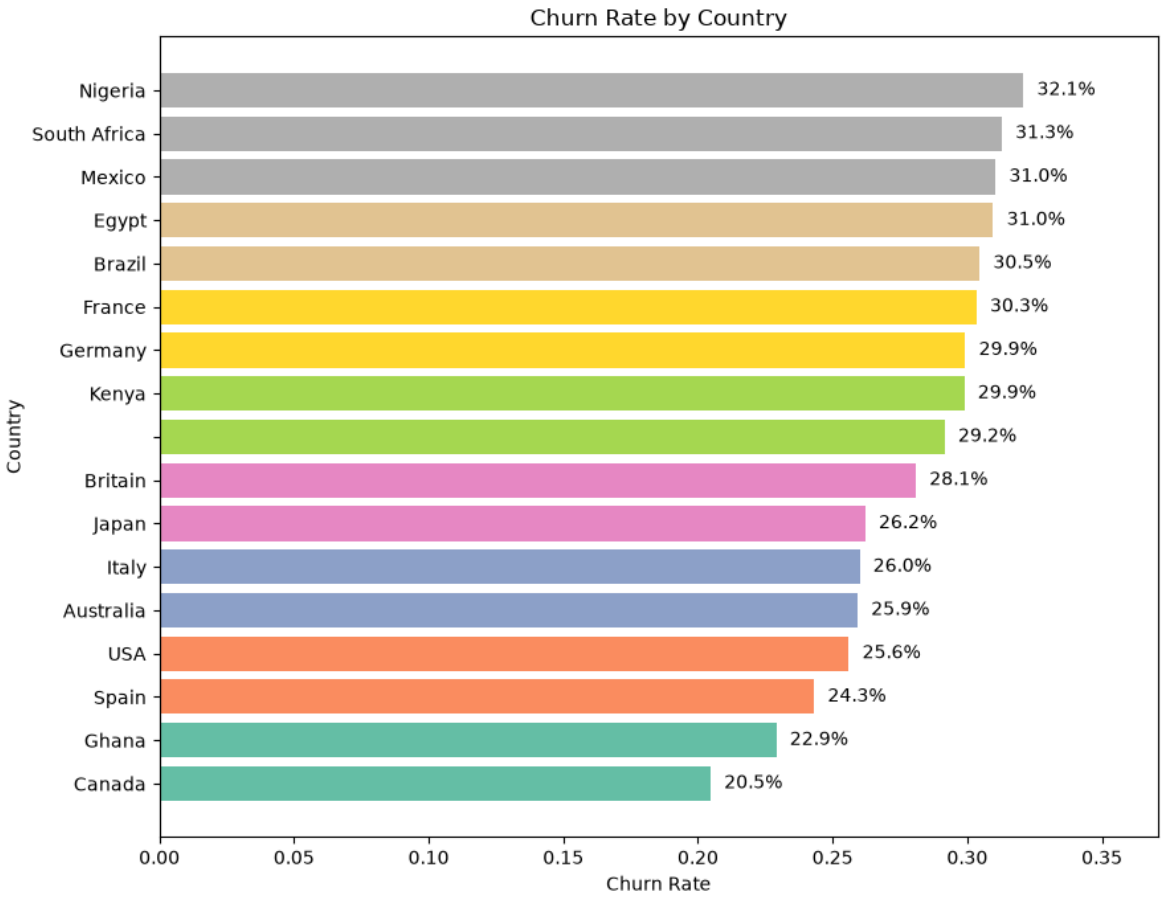
plt.figure(figsize=(9, 7))

bars = plt.barh(churn_country.index, churn_country.values, color=colors)

plt.xlabel('Churn Rate')
plt.ylabel('Country')
plt.title('Churn Rate by Country')

for bar, value in zip(bars, churn_country.values):
    plt.text(
        value + 0.005,
        bar.get_y() + bar.get_height()/2,
        f'{value:.1%}',
        va='center'
    )

plt.xlim(0, max(churn_country.values) + 0.05)
plt.tight_layout()
plt.show()
```



In [232…
```
churn_country
```

Out[232…
```
country
Canada          0.204678
Ghana           0.229050
Spain           0.243137
USA             0.256000
Australia       0.259259
Italy           0.260000
Japan           0.262097
Britain         0.280899
                0.291566
Kenya           0.298851
Germany         0.299270
France          0.303371
Brazil          0.304569
Egypt           0.309623
Mexico          0.310484
South Africa    0.312796
Nigeria         0.320833
Name: is_cancelled, dtype: float64
```

In [233…
```
# Correct method of encoding - based on priority
df_encoded = df2[['plan_type', 'contract_type', 'churn_score', 'is_cancelled', '

order_mappings = {
    'plan_type': ['Basic', 'Standard', 'Premium'],
    'contract_type': ['Monthly', 'Annual'],
    'Churn_risk': ['low', 'med', 'high']
}

for col, order in order_mappings.items():
```

```
    df_encoded[col] = pd.Categorical(
        df_encoded[col].astype('category'),
        categories=order,
        ordered=True
    ).codes
```

In [234…
```
df_encoded.head()
```

Out[234…

	plan_type	contract_type	churn_score	is_cancelled	Churn_risk	escalation_flag
0	2	0	54	0	1	0
1	2	0	81	1	2	1
2	2	1	5	0	0	0
3	1	0	26	0	0	0
4	1	1	28	0	0	0

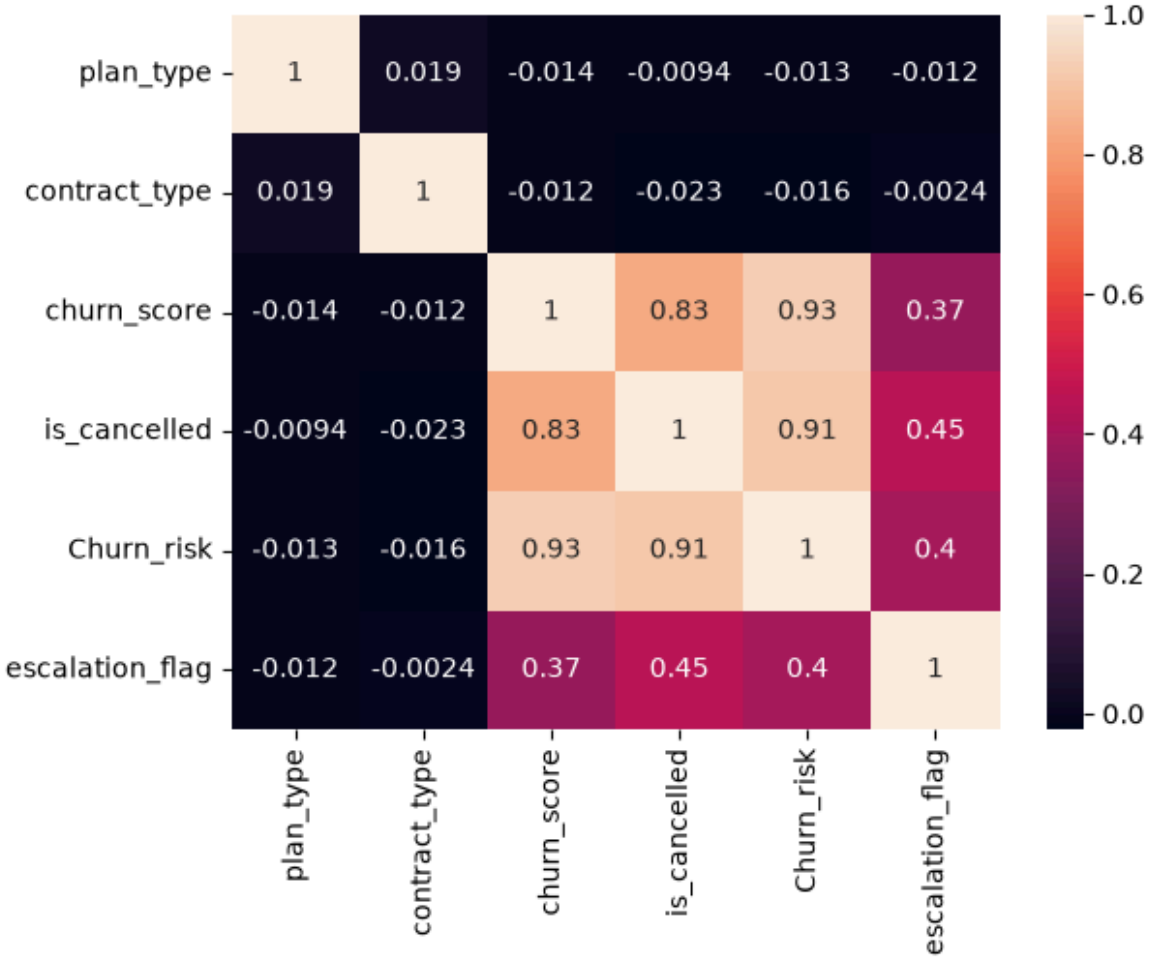
In [235…
```
df2[['plan_type', 'contract_type', 'churn_score', 'is_cancelled', 'Churn_risk',
```

Out[235…

	plan_type	contract_type	churn_score	is_cancelled	Churn_risk	escalation_flag
0	Premium	Monthly	54	0	med	0
1	Premium	Monthly	81	1	high	1
2	Premium	Annual	5	0	low	0
3	Standard	Monthly	26	0	low	0
4	Standard	Annual	28	0	low	0

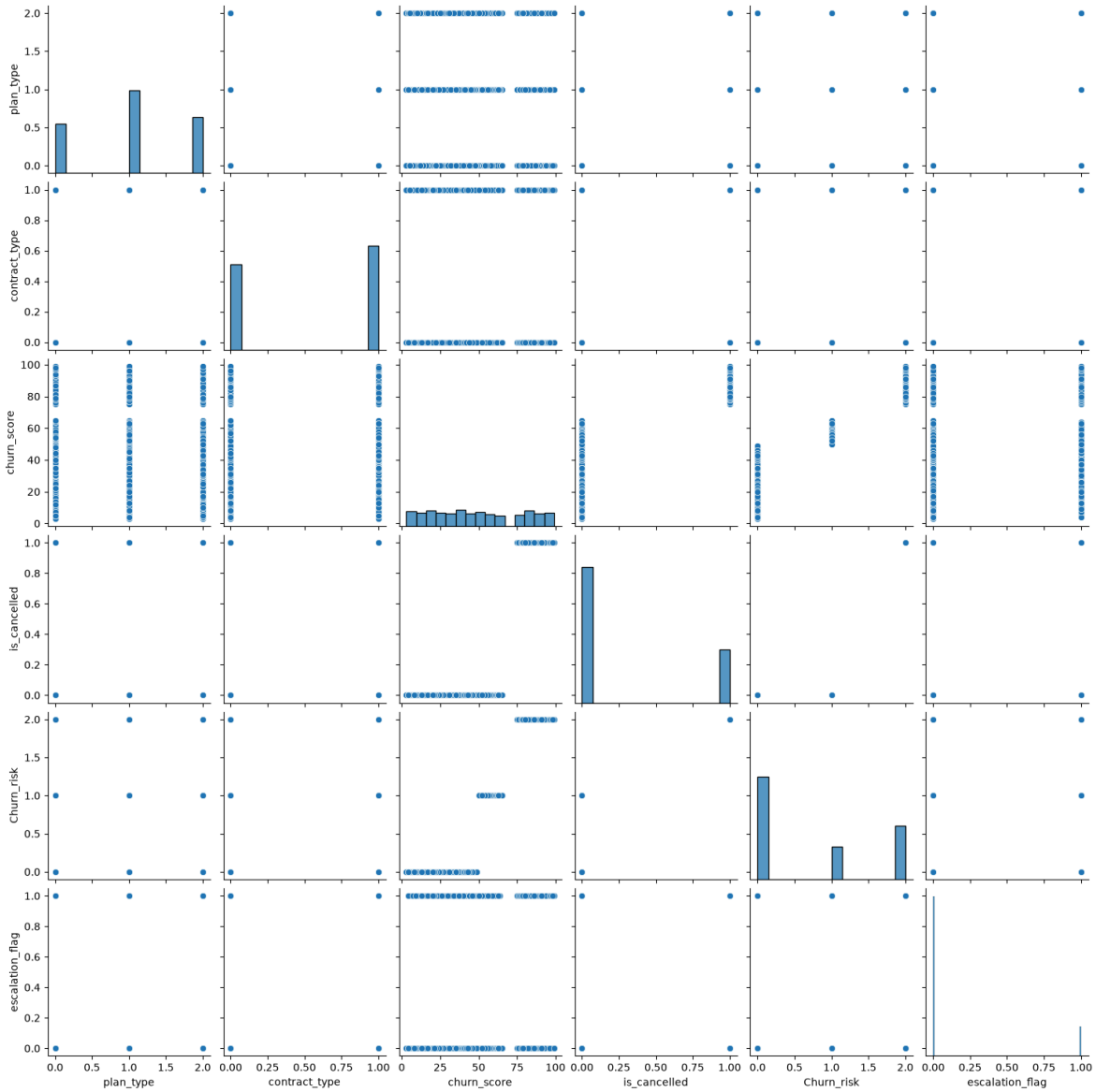
In [236…
```
sns.heatmap(df_encoded.corr(),annot=True)
```

Out[236…
```
<Axes: >
```



In [237…
```
sns.pairplot(df_encoded)
```

Out[237…
```
<seaborn.axisgrid.PairGrid at 0x1f8e4c16d50>
```



In [238…
```
pd.set_option('display.max.columns',100)
```

In [239…
```
df.head()
```

Out[239…

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
0	0002-ORFBO	2020-05-14	Refferal	2025-05-14	Premium	
1	0003-MKNFE	2020-03-19	Refferal	2024-03-19	Premium	
2	0004-TLHLJ	2020-02-16	Refferal	2025-02-16	Premium	
3	0011-IGKFF	2020-11-25	Organic	2024-11-25	Standard	
4	0013-EXCHZ	2022-05-12	Paid	2024-05-12	Standard	

In [240…
```
arpu = df['monthly_charges'].mean()
print("ARPU:", round(arpu, 2))
```

```
ARPU: 16.81
```

In [241…
```
total_revenue = df['monthly_charges'].sum()
print("Total Monthly Revenue:", round(total_revenue, 2))
```

```
Total Monthly Revenue: 70381.12
```

In [242…
```
revenue_loss = df.loc[
    df['is_cancelled'] == 1,
    'monthly_charges'
].sum()

print("Revenue Lost Due to Churn:", round(revenue_loss, 2))
revenue_loss_pct = (
    revenue_loss / total_revenue
) * 100

print("Revenue Loss %:", round(revenue_loss_pct, 2), "%")
```

```
Revenue Lost Due to Churn: 19483.21
Revenue Loss %: 27.68 %
```

In [243…
```
priority_customers = df[
    (df['Churn_risk'] == 'high') &
    (df['is_cancelled'] == 0)
].sort_values(
    by=['cltv', 'monthly_charges'],
    ascending=False
)

priority_customers[
    [
        'customerid',
        'Customer Name',
        'plan_type',
        'contract_type',
        'monthly_charges',
        'cltv',
        'churn_score',
        'Churn_risk'
    ]
].head(10)
```

Out[243…

	customerid	Customer Name	plan_type	contract_type	monthly_charges	cltv	churn_score

In [247…
```
df[df['Churn_risk'] == 'high'].head()
```

Out[247...

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cor
1	0003-MKNFE	2020-03-19	Refferal	2024-03-19	Premium	
5	0013-MHZWF	2023-05-25	Refferal	2024-05-25	Premium	
9	0016-QLJIS	2019-05-19	Paid	2025-05-19	Premium	
15	0020-INWCK	2022-07-10	Organic	2025-07-10	Standard	
17	0021-IKXGC	2022-12-25	Organic	2024-12-25	Standard	

In [248...

```
mrr = df['monthly_charges'].sum()

print("MRR:", round(mrr, 2))
```

```
MRR: 70381.12
```

In [250...

```
total_cltv = df['cltv'].sum()
avg_cltv = df['cltv'].mean()

print("Total CLTV:", round(total_cltv, 2))
print("Average CLTV:", round(avg_cltv, 2))
```

```
Total CLTV: 4465882
Average CLTV: 1066.35
```

In [252...

```
risk_customers = df['Churn_risk'].value_counts()
print("Risk Customers:",risk_customers)
```

```
Risk Customers: Churn_risk
low     2279
high    1179
med      730
Name: count, dtype: int64
```

In [253...

```
high_risk = df[df['Churn_risk'] == 'high']

print("High-Risk Customers:", high_risk['customerid'].nunique())
print("High-Risk MRR:", round(high_risk['monthly_charges'].sum(), 2))
print("High-Risk CLTV:", round(high_risk['cltv'].sum(), 2))
print("High-Risk % of Customers:",
      round(high_risk['customerid'].nunique() / df['customerid'].nunique() * 100
```

```
High-Risk Customers: 1179
High-Risk MRR: 19483.21
High-Risk CLTV: 730645
High-Risk % of Customers: 28.15
```

In [254...

```
df['cancellation_reason'].value_counts()
```

Out[254…

```
cancellation_reason
                            3009
Too expensive                264
Poor streaming quality       240
Switched to competitor       229
Forgot to cancel trial       227
Not enough content           219
Name: count, dtype: int64
```

In [ ]: